Cloud Architecture (AWS/Azure)

From Model to Operated Product — Course Reader

Both session lectures of the one-day intensive, with sample homework problems and exam questions.

Topics Covered

building-blocks The building blocks
designing-for-real-use Designing for real use
reasoning-primer-subject How to reason about cloud-architecture cases
reasoning-primer-general Core reasoning skills

Topic building-blocks: The building blocks

The building blocks

Cloud computing means renting capacity instead of owning hardware. AWS or Azure operates the physical facilities, but the customer still chooses the architecture, configures access, protects data, and pays for what runs. The ownership boundary changes; responsibility does not disappear.

Most cloud services are variations on four primitives: compute, storage, network, and identity. Compute forms a ladder from virtual machines, to containers, to serverless functions. Moving up the ladder reduces operating work but also reduces control over the runtime.

Storage is not one interchangeable bucket. Object storage suits durable files and blobs, block storage behaves like a disk attached to a machine, and file storage gives several systems a shared hierarchy. Managed relational and key-value databases remove routine infrastructure work, but schema, indexing, capacity, backups, and access remain design responsibilities.

Geography also shapes the architecture. A region is a geographic market; availability zones are isolated locations within it. A virtual private cloud or virtual network, subnets, routes, and security groups control how resources communicate. Identity and access management controls who or what may act. Least privilege matters because a broad permission can expose an otherwise well-written system.

Topic designing-for-real-use: Designing for real use

Designing for real use

A cloud diagram becomes an architecture only when it explains how the system behaves under traffic, failure, recovery, and change. Scaling up gives one machine more capacity; scaling out adds machines. Horizontal scaling usually wins for resilience, but it requires a load balancer and application instances that do not depend on local session state.

Designing for failure means assuming instances, zones, dependencies, and deployments will break. Redundancy keeps service available; backups preserve data; recovery objectives define how much data loss and downtime the business can tolerate. A backup that has never been restored is only a hope.

Pay-per-use makes cost a design constraint. Managed services can reduce operational work, while self-hosting can preserve control and portability; neither is automatically cheaper. Infrastructure as code makes the architecture reviewable and repeatable. Logs, metrics, and traces make behavior visible.

Security follows the shared responsibility model. Encrypt data, protect secrets, minimize permissions, patch what you own, and test recovery. A stateless application spread across zones can survive a traffic spike and an instance loss; a single stateful server with no tested backup may fail even when its code is correct.

Topic reasoning-primer-subject: How to reason about cloud-architecture cases

How to reason about cloud-architecture cases

This primer prepares you for the Cloud Architecture diagnostic. It is ungraded, fresh on every attempt, and asks you to apply the two sessions rather than recite terms.

It tests application, not memorization

A question gives a short scene: a public database is reachable from the internet, a workload has account-wide permissions, a load-balanced application loses local sessions, or an untested nightly backup cannot meet a ten-minute recovery point. Name the cloud-architecture idea that explains the failure and point to the scene detail that decides it.

What the questions reward • Naming the right idea — distinguish compute, storage, network, and identity choices; vertical and horizontal scaling; redundancy and backup; recovery objectives; cost, observability, and shared responsibility. • Using evidence from the scene — point to the detail in the situation that supports your answer, rather than answering from a general impression. • Choosing the right operating layer — decide whether the remedy belongs in resource selection, network boundaries, permissions, state management, failure recovery, infrastructure as code, observability, or security.

How to do this activity well

1. Read the situation first, then ask which topic it belongs to. 2. Find the detail that decides it — what in the scene makes one answer better than another. 3. For written items, give the core idea in a sentence or two — clear and correct beats long and padded.

Take it as often as you like; the questions are freshly generated every time, and there is no penalty for any answer.

Topic reasoning-primer-general: Core reasoning skills

Core reasoning skills

This short primer prepares you for the General Reasoning diagnostic — an ungraded, fresh-attempt check of five genuine reasoning skills. The skills remain domain-general, with cloud-architecture examples showing how they guide engineering judgment.

The five skills • Analysis separates layers and assumptions. A timeout caused by a saturated database is different from one caused by an unhealthy application instance. • Inference distinguishes what a signal proves from what it merely suggests. Rising CPU may justify investigation; by itself it does not prove horizontal scaling is the only solution. • Evaluation asks whether evidence supports a decision. A successful backup job cannot justify a recovery claim without a measured restore. • Deduction identifies architectural guarantees. If every request can reach any stateless instance and durable state is shared, replacing one instance does not erase a session. • Induction generalizes cautiously from samples. One successful failover test supports limited confidence; repeated tests under representative load support a stronger claim.

A recurring trap: things that move together

Most wrong answers sound plausible but outrun the evidence. Keep apart what facts show, what architecture guarantees, what you assume, and what is only possible. A multi-zone diagram suggests redundancy; a measured zone-failure exercise is stronger evidence that the system actually recovers.

How to do this activity well

1. Find the point (conclusion) first, then the reasons. 2. Ask which of the five skills the question is testing (a hidden-assumption question is analysis; a "what follows" question is inference or deduction; a "how good is this reasoning" question is evaluation). 3. Pick the option that follows only from what you were given — not the one that merely sounds true or appealing.

Take it as often as you like; the questions are freshly generated every time, and it never affects your grade.

Practice Homework (Sampler)

Problem 1 (Topic building-blocks)
Northstar Media stores a growing video archive on premium block volumes attached to one virtual machine. Costs are rising, uploads stop whenever that machine is replaced, and viewers only need to fetch complete video files. Recommend a better storage design and explain why it fits the access pattern. (3–5 sentences.)

Problem 2 (Topic building-blocks)
Priya deploys a payroll service on a public virtual machine whose role can read every storage bucket and secret in the account. The application needs one private database and one payroll-export bucket. Redesign its network and identity boundaries using least privilege. (3–5 sentences.)

Problem 3 (Topic designing-for-real-use)
Harbor Tickets runs two web servers behind a load balancer, but each server keeps customer carts in local memory. During a launch, scaled-out instances send returning customers to different servers and their carts disappear. Explain the architectural failure and redesign the state handling. (3–5 sentences.)

Problem 4 (Topic designing-for-real-use)
Mateo's team copies its production database into a second availability zone and calls disaster recovery complete, but it has never restored a backup or measured failover. The business allows at most 15 minutes of data loss and one hour of downtime. Design a testable recovery plan. (3–5 sentences.)

Sample Exam Questions

Question 1 (Final — Cloud Architecture)
A research publisher is moving an internal manuscript platform to AWS or Azure. It needs a public web entry point, private application and database tiers, durable document storage, controlled staff access, and continued service through one availability-zone failure. Trace a defensible architecture from the four primitives through regional placement and permissions. (5–7 sentences.)

Question 2 (Course Test — Cloud Architecture)
Atlas Health must place a patient portal in one cloud region. Users need a public HTTPS endpoint, but application instances and the relational database must not be directly reachable from the internet. Design the network, identity, and availability-zone layout. (4–6 sentences.)

Question 3 (Course Test — Cloud Architecture)
A legal-document service promises recovery within two hours with no more than ten minutes of lost uploads. It has multi-zone compute but keeps one database and has nightly backups that nobody has restored. Explain the gaps and design recovery evidence that would justify the promise. (4–6 sentences.)

Answers, AI tutoring, adaptive practice, and graded assignments are available in the online Cloud Architecture (AWS/Azure) course.